

Traspaso de datos · stack Python (septiembre 2026)

Guía para el agente de la rama `python-experiment`. **Cero escrituras a la BD de producción en todo este traspaso: solo lectura + archivos.**

0. Decisión tomada

| Punto | Decisión | --- | --- | | Fuente de verdad | **prod** (soporte-postgres), NO dev | |
soporte-postgres-dev | **muere** · no hay que sanear nada ahí | IDs en la instancia nueva | **desde 1** (se reasignan; no se preservan los de prod) | | Tilde | **CON tilde**: Académicos Modular Académicos Semestral | |
Alcance | solo **septiembre** (253 filas) | |

1. Contexto bloqueante: hay DOS instancias distintas

El análisis previo ("el corte es del 2026-08-31 en adelante, 25 filas") es correcto **solo para dev**. Producción es otra instancia y sus números son completamente distintos.

| | **prod** (soporte-postgres) | **dev** (soporte-postgres-dev) | | --- | --- | --- | | Contenedor |
soporte-postgres | soporte-postgres-dev | | IP en su red | 10.0.13.2 | 10.0.18.2 | | Red docker |
soporte_tecnico_soporte-network | soporte_tecnico_dev-network | | Puerto en host | **ninguno**
· 5433 | | DB / user | soporte / soporte | soporte / soporte | | Backend / frontend | :5000 / :3002 |
:5001 / :3003 | | Atenciones | 2191 | 1957 | | Septiembre | 253 | 18 | | Areas / Grupos / GruposPadres | 0 / 0
/ 3 | 53 / 4 / 3 | | Sin jerarquía | 2191 / 2191 (100%) | 25 (ago 7 + sep 18) | | Horarios sept 2026 | 2 | 0 | |
alembic_version | · no existe | 0001_baseline | |

Prod no tiene el puerto mapeado al host. Se accede con `docker exec soporte-postgres psql -U soporte -d soporte` o desde dentro de la red. Si necesitas conectarte desde el host, hay que abrir un `port-forward` · no lo hagas sin permiso.

Los dos "septiembre" NO son el mismo dataset

| | Filas | Descripciones compartidas | IDs | | --- | --- | --- | --- | | dev septiembre | 18 | 3 / 18 | · | prod septiembre | 253 | · | **los 18 IDs colisionan con contenido distinto** | |

Id 1945: dev="Se solicito brindar internet por cable a" |
prod="Requieren Wifi docentes de medicina"
Id 1946: dev="Problemas con la impresora del ticketero" |
prod="Problemas para iniciar sesion en usuario"

15 de 18 filas de dev **no existen** en prod. **No se pueden mezclar por ID.** La fuente es prod.

2. Hechos verificados (no re-derivar)

Sobre las 253 filas de septiembre de **prod**:

| Dato | Valor | | --- | --- | | Filas | 253 (2026-09-01 - 2026-09-19) | | AreaSolicitante distintos | 48 | |
Resueltos por mapeo-areas.csv | 253 / 253 · 0 sin mapear | | Áreas canónicas usadas | 32 / 53 | | Técnicos distintos |
7 · todos en SEED_USUARIOS | | Colaboradores distintos | 8 · todos en SEED_USUARIOS | | Filas con colaborador |
45 | | FueraDeTurno = true | 7 | | Categorías | las 8 válidas · Medios | Interno, Presencial, WhatsApp ·
los 3 válidos | | UsuarioSolicitante | ADM 150, BEC 40, EST 36, DOC 27 · los 4 válidos · | |

EIAG (6 filas) **ya fue migrado a ADM** en el CSV, siguiendo el precedente del proyecto (`usuarioSolicitante`

bajó de 5 a 4 valores). El CSV no contiene valores fuera de `SOLICITANTES_VALIDOS = {ADM, BEC, DOC, EST}`.

3. Artefactos entregados

Todos en la raíz de este repo, ya normalizados **con tilde**:

| Archivo | Contenido | |---|---| | `atenciones_septiembre.csv` | 253 filas × 13 columnas · las atenciones de prod, con area ya **canónica** | | `mapeo-areas-dedup.csv` | Catálogo: 53 áreas (grupo_padre, grupo, nombre, activo) | | `mapeo-areas.csv` | Regla: 144 orígenes · 53 destinos (area_actual, grupo_padre, grupo, nuevo_nombre, activo, notas) |

Columnas de `atenciones_septiembre.csv`

`tecnico_email, area, colaborador_email, medio_solicitud, usuario_solicitante, categoria, descripcion, solucion, observaciones, enlace_apoyo, fuera_de_turno, fecha_registro, created_at`

- `area` = nombre **canónico** del catálogo (la columna `AreaSolicitante` original de prod ya está resuelta aquí)
- `tecnico_email / colaborador_email` = **emails**, no IDs (estables; los IDs se reasignan)
- `fuera_de_turno` = "true" / "false"
- **Sin Id** · que la instancia nueva los asigne desde 1

Cambios aplicados a los CSV de mapeo

Se normalizó la tilde en la columna destino **95 celdas**, 0 en columnas equivocadas):

`Academicos Modular` · Académicos Modular
`Academicos Semestral` · Académicos Semestral

Motivo: `prod.AreaSolicitante` dice Académicos Modular **con tilde**. Con el catálogo en tilde, el match exacto funciona y esas 2 áreas dejan de ser caso especial.

Nota: `mapeo-areas.csv` del repo es **más nuevo** que el de `/home/mattias/Personal/` (asigna Grupo a Contabilidad y Vicerrectorado Administrativo). Use el del repo.

4. Catálogo a sembrar

· Administrativos
 · Investigación (5) · Rectorado (5) · Vicerrectorado (5) · (sin grupo) 24
· Académicos
 · (sin grupo) (3)
· Extras
 · Eventos (2) · (sin grupo) 9

- 3 GruposPadres · 4 Grupos · 53 Áreas **todas activas**
 - 17 áreas con Grupo · 36 sin Grupo · ya cumplida área tiene `GrupoPadre`, aunque no tenga `Grupo`
 - `Areas.GrupoPadreId` es NOT NULL; `Areas.GrupoId` es nullable · no hay que inventar el nivel intermedio
-

5. · BUG A CERRAR (bloqueante, es la causa raíz)

Sin esto, el saneo no sirve: van a seguir naciendo filas en el limbo.

Archivo: backend-fastapi/app/routers/atenciones.py, función_resolver_jerarquia (líneas 126-160)

```
    if legacy:
        area = db.scalars(select(Area).where(Area.nombre ==
legacy)).first()    # · 144: match EXACTO
        if area is not None:
            return area.grupo_padre_id, area.grupo_id, area.id,
legacy
        grupo = db.scalars(select(Grupo).where(Grupo.nombre ==
legacy)).first()
        ...
        if gp_id is None and not legacy:
            raise bad_request(...)
        if not legacy and gp_id is not None:
            ...
        return gp_id, g_id, ar_id, legacy    # · 160: cae aquí con los 3
en NULL, EN SILENCIO
```

Dos defectos:

- 1 **Línea 144** comparación exacta "Académicos Modular" vs "academicos modular " no matchea
- 2 **Línea 160** si nada matchea **no falla: devuelve los 3 FK en NULL**. La fila nace en el limbo.

Fix propuesto

```
import unicodedata
def _norm(s: str) -> str:
    """minúsculas + sin tildes + espacios colapsados (robusto a
variantes históricas)"""
    s = unicodedata.normalize("NFKD", s)
    s = "".join(c for c in s if not unicodedata.combining(c))
    return " ".join(s.lower().split())
def _buscar_area(db: DbSession, texto: str) -> Area | None:
    """Match exacto primero; si falla, match normalizado contra
todo el catálogo."""
    a = db.scalars(select(Area).where(Area.nombre ==
texto)).first()
    if a is not None:
        return a
    objetivo = _norm(texto)
    return next(
        (x for x in
db.scalars(select(Area).where(Area.activo.is_(True))).all()
        if _norm(x.nombre) == objetivo),
        None,
    )
```

Y en `_resolver_jerarquia`, reemplazar el `fallthrough` de la línea 160 por:

```
# nada matcheó: cerrar la puerta en vez de crear una fila en el
limbo
raise bad_request(
    f'"{legacy}" no existe en el catálogo
(Areas/Grupos/GruposPadres). '
    "Envíe un area_id válido."
)
```

Con los CSV ya normalizados, el match exacto alcanza. La normalización es defensa contrados 104 textos históricos distintos.

6. Extensión propuesta para `scripts/seed.py`

`seed.py` ya siembra **9 usuarios + 3 GruposPadres** con los mismos IDs de prod, es idempotente y usa los modelos de la app. **Extenderlo, no crear SQL suelto.**

```
# --- agregar a scripts/seed.py ---
import csv
from datetime import date
from pathlib import Path
from sqlalchemy import select
from app.models.area import Area
from app.models.atencion import Atencion
from app.models.grupo import Grupo
RAIZ = Path(__file__).resolve().parents[2] #
Soporte_Tecnico-python/
CATALOGO = RAIZ / "mapeo-areas-dedup.csv"
ATENCIONES = RAIZ / "atenciones_septiembre.csv"
def _leer(path: Path) -> list[dict[str, str]]:
    with open(path, encoding="utf-8", newline="") as f:
        return list(csv.DictReader(f))
def seed_catalogo(db) -> dict[str, int]:
    """4 Grupos + 53 Áreas desde mapeo-areas-dedup.csv. Idempotente
    por nombre."""
    filas = _leer(CATALOGO)
    padres = {p.nombre: p.id for p in
db.scalars(select(GrupoPadre)).all()}
    stats = {"grupos": 0, "areas": 0}
    for padre, grupo in dict.fromkeys(
        (f["grupo_padre"].strip(), f["grupo"].strip()) for f in
    filas
    ):
        if not grupo:
            continue
        pid = padres[padre]
        if db.scalars(
            select(Grupo).where(Grupo.nombre == grupo,
Grupo.grupo_padre_id == pid)
        ).first() is None:
            db.add(Grupo(nombre=grupo, grupo_padre_id=pid,
```

```

activo=True))
        stats["grupos"] += 1
    db.flush()
    gids = {(g.grupo_padre_id, g.nombre): g.id for g in
db.scalars(select(Grupo)).all()}
    for f in filas:
        pid = padres[f["grupo_padre"].strip()]
        nombre = f["nombre"].strip()
        if db.scalars(
            select(Area).where(Area.nombre == nombre,
Area.grupo_padre_id == pid)
        ).first() is not None:
            continue
        grupo_nombre = f["grupo"].strip()
        db.add(
            Area(
                nombre=nombre,
                grupo_padre_id=pid,
                grupo_id=gids.get((pid, grupo_nombre)) if
grupo_nombre else None,
                activo=f["activo"].strip() == "1",
            )
        )
        stats["areas"] += 1
    db.flush()
    return stats
def seed_atenciones(db) -> dict[str, int]:
    """253 atenciones de septiembre. Deriva los 3 FK desde el área
(no se teclea la jerarquía)."""
    filas = _leer(ATENCIONES)
    usuarios = {u.email: u.id for u in
db.scalars(select(Usuario)).all()}
    areas = {a.nombre: a for a in db.scalars(select(Area)).all()}
    ya = db.scalar(select(func.count()).select_from(Atencion)) or 0
    if ya:
        return {"atenciones": 0, "omitido": f"ya hay {ya}
atenciones"}
    stats = {"atenciones": 0, "con_colaborador": 0,
"fuera_de_turno": 0}
    for f in filas:
        area = areas.get(f["area"].strip())
        if area is None:
            raise RuntimeError(f"Área '{f['area']}' no existe en el
catálogo")
        email = f["tecnico_email"].strip()
        if email not in usuarios:
            raise RuntimeError(f"Técnico '{email}' no existe en
Usuarios")
        colab_email = f["colaborador_email"].strip()
        fuera = f["fuera_de_turno"].strip() == "true"
        db.add(
            Atencion(
                usuario_id=usuarios[email],
                grupo_padre_id=area.grupo_padre_id, # · derivado

```

```

del área
        grupo_id=area.grupo_id,                # · derivado
del área
        area_id=area.id,
        area_solicitante=area.nombre,          # texto
legado ya normalizado
        medio_solicitud=f["medio_solicitud"],
        usuario_solicitante=f["usuario_solicitante"],
        categoria=f["categoria"],
        descripcion=f["descripcion"],
        solucion=f["solucion"],
        observaciones=f["observaciones"] or None,
        enlace_apoyo=f["enlace_apoyo"] or None,
        colaborador_id=usuarios.get(colab_email) if
colab_email else None,
        fuera_de_turno=fuera,

        fecha_registro=date.fromisoformat(f["fecha_registro"]),
        created_at=datetime.fromisoformat(f["created_at"]),
    )
    )
    stats["atenciones"] += 1
    stats["con_colaborador"] += 1 if colab_email else 0
    stats["fuera_de_turno"] += 1 if fuera else 0
    db.flush()
    return stats

```

Y en `run_seed()`, antes del `db.commit()`:

```

stats |= seed_catalogo(db)
stats |= seed_atenciones(db)

```

7. Pasos de ejecución

```

cd /home/mattias/Proyectos/Soporte_Tecnico-python/backend-fastapi
# 0) BD limpia
alembic upgrade head
# 1) catálogo + usuarios + atenciones (un solo comando, idempotente)
python scripts/seed.py

```

Orden interno: GruposPadres · Usuarios · Grupos · Áreas · ~~Atenciones~~ **seed.py** ya lo respeta.

8. Gotchas (8.1 ya está resuelto)

8.1 EIAG en UsuarioSolicitante · · RESUELTO

Las 6 filas con EIAG **ya fueron migradas a ADM** en `atenciones_septiembre.csv`. El CSV quedó con ADM 150 · BEC 40 · DOC 27 · EST (263 total), todos dentro de `SOLICITANTES_VALIDOS = {ADM, BEC, DOC, EST}`.

No hace falta ninguna acción adicional (Si en el futuro entran datos viejos con EIAG, la normalización ya está

reflejada en el CSV; el chequeo de §9 valida que no queden.)

8.2 Horarios · opcional

FueraDeTurno es un **bool guardado** (7 en true), así que las atenciones no dependen de Horarios para importar. Solo hace falta si el cálculo se va a recalcular. Prod tiene 2 horarios de septiembre.

8.3 AreaSolicitante se conserva

Se mantiene la columna como texto (ahora con el nombre canónico) para trazabilidad ~~además~~ del FK. Si prefieres eliminar la redundancia, quita area_solicitante del modelo y del seed.

8.4 Los 3 FK son redundantes

Atencion.area_id es lo único necesario grupo_id y grupo_padre_id ya viven en Areas. Se guardan por compatibilidad con el esquema actual. Si rediseñas, considera dejasolo area_id y derivar el resto por join · evita que se desincronicen.

9. Verificación (correr después del seed)

```
-- deben dar exactamente estos números
SELECT COUNT(*) FROM "Atenciones";
-- 253
SELECT COUNT(*) FROM "Atenciones" WHERE "AreaId" IS NULL;
-- 0
SELECT COUNT(*) FROM "Atenciones" WHERE "ColaboradorId" IS NOT
NULL; -- 45
SELECT COUNT(*) FROM "Atenciones" WHERE "FueraDeTurno";
-- 7
SELECT COUNT(*) FROM "Areas";
-- 53
SELECT COUNT(*) FROM "Grupos";
-- 4
SELECT COUNT(*) FROM "GruposPadres";
-- 3
-- valores fuera de los catalogos validos (los 3 deben dar 0)
SELECT COUNT(*) FROM "Atenciones"
WHERE "UsuarioSolicitante" NOT IN ('ADM','BEC','DOC','EST');
-- 0
SELECT COUNT(*) FROM "Atenciones"
WHERE "MedioSolicitud" NOT IN
('Presencial','Interno','WhatsApp','E-ticket'); -- 0
SELECT COUNT(*) FROM "Atenciones"
WHERE "Categoria" NOT IN
('Audio/Video','Cuentas/Accesos','Hardware','Impresión',
'Otros','Redes/Conectividad','Sistemas académicos','Software');
-- 0
-- integridad jerarquica: toda atencion con area debe tener los 3
FK coherentes
SELECT COUNT(*) FROM "Atenciones" a
JOIN "Areas" ar ON ar."Id" = a."AreaId"
WHERE a."GrupoPadreId" IS DISTINCT FROM ar."GrupoPadreId"
OR a."GrupoId" IS DISTINCT FROM ar."GrupoId";
```

-- 0

Checks de porcentaje:

```
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE "AreaId" IS NOT NULL) /  
COUNT(*), 1)  
FROM "Atenciones";    -- 100.0
```

10. Resumen de lo que se tocó

| Archivo | Cambio | |---|---| | mapeo-areas-dedup.csv | 2 celdas: tilde en nombre | | mapeo-areas.csv |
93 celdas: tilde en nuevo_nombre | | atenciones_septiembre.csv | **nuevo** · 253 filas de prod area
canónica +EIAG-ADM (6) | | docs/traspaso-datos-septiembre.md | **nuevo** · este documento |

Nada en backend-fastapi/ fue modificado. El fix de `_resolver_jerarquia` (§5) y la extensión
`seed.py` (§6) quedan como propuesta para que los apliques tú.